LOGBRANIK

FROM LOGS TO CONTROLLED ACTION

NGINX LOG SECURITY

Build a local IP guard, signed policies, and a reliable analysis pipeline



Aleksandar Popovic

NGINX OPEN SOURCE / PYTHON LAB

OPEN BOOK: CC BY 4.0 / CODE: MIT

Contents

Nginx Log Security 3

1. Understand the feedback loop 7

2. Draw the trust boundaries 10

3. Design around failure 13

4. Run the laboratory 16

CC BY 4.0 / code MIT - candidate 0.2

Nginx Log Security

Build a local IP guard, signed policies, and a reliable analysis pipeline

LogBranik - From logs to controlled action

Author: Aleksandar Popovic

Open-source publication candidate, edition 0.2. Prepared October 2026.

Copyright and open licenses

Copyright (c) 2026 Aleksandar Popovic. The complete book text, exercises, answers, prompts, and original LogBranik visual assets are licensed under Creative Commons Attribution 4.0 International. The complete CC BY 4.0 legal code governs that grant. Original code, code snippets and configuration examples are licensed under the MIT License. The source package includes the full MIT notice and a detailed licensing map.

You may share, modify and redistribute these materials, including commercially, under the applicable license. Attribute content to Aleksandar Popovic, link CC BY 4.0, and identify changes. Retain the MIT copyright and permission notice with code copies. Third-party fonts and dependencies retain their own licenses. An optional paid reading edition does not restrict access to the open sources. This edition has not yet been published to a public repository.

MIT notice for original code

MIT License. Copyright (c) 2026 Aleksandar Popovic.

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Introduction

An ordinary Nginx server already knows a great deal about the traffic reaching an application. Its access logs tell us which paths clients request, how the application responds, and how much work each request causes. Turning those observations into an automatic defense is possible without buying Nginx Plus. The difficult part is deciding what the observations mean, delivering a trustworthy decision, and keeping the application usable when a component fails.

This book builds that system from explicit contracts. An edge host keeps request checks local. A central service receives completed access events, identifies a narrow pattern of suspicious activity, and publishes short-lived IP decisions. An agent accepts signed policies and answers Nginx authorization subrequests. The design distinguishes an operator's intention from what an edge has actually applied.

The companion implementation is deliberately compact. Python and SQLite let you inspect persistence, deduplication, detection, signatures, revision ordering, and expiry without first operating several infrastructure services. Its live transport exercises use mutually authenticated HTTPS. The deployment design then extends those ideas to Vector, FastAPI, PostgreSQL, and a Go agent. That production design is a specification and migration path, not a claim that the small laboratory is production software.

Who should read this book

You should be comfortable with Linux processes, HTTP, a basic Nginx configuration, and reading Python. You do not need to know public-key cryptography. The relevant operations are introduced through a concrete policy protocol. The PostgreSQL and Go lessons assume that you can read SQL and understand a simple service boundary; they explain the architecture rather than teach either language from scratch.

Work on servers you own or have permission to administer. All example addresses use documentation ranges or loopback. The traffic exercises target a local echo application. They are designed to study defensive behavior without contacting another organization's systems.

What you will build

The laboratory includes a durable ingestion store, an explainable 404-path detector, a signed full-state snapshot, a local authorization handler, a separate mTLS control handler, and a periodic delivery process. You will observe allow, would-block, block, and revoke transitions. You will test old-policy rejection, duplicate ingestion, clock changes, and expiry without a central server.

Each of the twenty lessons ends with an exercise and an answer. The source of every lesson is a Markdown file in `lessons/`. The same content is exported to Leanpub manuscript files. The appendices contain a runbook, a bridge to the sixty-task implementation pack, a glossary, and source references.

Evidence and edition status

The delivered verification directory records the actual Python test run and environment. The core and live loopback mTLS tests were executed during preparation. The host does not support creating an AF_UNIX socket, so that integration test is explicitly skipped. Nginx, Vector, PostgreSQL, and Go were unavailable in the preparation environment. Their configuration examples and production exercises therefore require execution on the reader's deployment host. No end-to-end Nginx performance measurement or production readiness claim is made.

This distinction is part of the engineering method. A plausible configuration is not a verified deployment. A unit test of a policy function is not a test of an Nginx access phase. A successful loopback HTTPS exchange is not a fleet rollout. The acceptance gates in later lessons tell you which evidence belongs to which claim.

AI transparency

AI was used to generate the manuscript, companion code, exercises, and publishing materials. Automated checks were executed for the scope documented above. This candidate has not yet received an independent human technical review or an author's final editorial approval. Before commercial publication, the responsible author should review the text, reproduce the deployment exercises, and update this statement to describe the work actually completed. Do not replace this statement with a claim that AI was used only for proofreading.

Reading route

Read lessons 1-3 for the system model. Run lesson 4 before changing a server. Lessons 5-12 cover trustworthy data and control. Lessons 13-17 cover operations and the production migration. Lessons 18-20 provide testing, troubleshooting, and a reproducible release workflow. The appendix runbook is useful at the keyboard, but it does not replace the explanations.

LogBranik is a proposed project brand. It is not an official Nginx product and does not imply endorsement by Nginx, Leanpub, or any dependency author. The name has not received trademark clearance. There is no fabricated author biography, endorsement, benchmark, or customer story in this edition.

1. Understand the feedback loop

Start with the event that actually exists

Imagine that a client requests five nonexistent paths from your application. Nginx proxies each request, the application returns 404, and Nginx writes an access record. An analyzer can now see five completed observations. It cannot travel backward and prevent those requests. Its first useful action affects a later request from the same address.

That timing determines the system's scope. Access-log analysis can interrupt repeated activity after evidence accumulates. It does not inspect a request before it starts, and it does not terminate an already established stream. A slow response delays the corresponding access event. A long-lived connection may not produce its terminal record for a long time. Network capacity can be exhausted before this application-level loop has enough evidence.

Instead of promising instant protection, define a measurable reaction interval. Start it when the detector has enough eligible events to create a decision. End it when the edge has persisted and published the policy containing that decision. Separately measure the delay from request completion to ingestion. Those two intervals reveal whether collection or control delivery is the bottleneck.

Three different things called a block

An IP decision in a database is desired state. A signed message containing it is delivery state. A revision published by an agent is applied state. Calling all three "blocked" makes an operator dashboard misleading. A central service can create a ban while an edge remains disconnected. An HTTP sender can finish transmitting while the receiver rejects an epoch mismatch. An agent can accept a policy whose decisions are already expired.

The first vocabulary rule is to name the state precisely. A decision may be active centrally and unapplied locally. A delivery may be pending, retrying, or acknowledged. A local policy may be applied,

but contain no active bans. The edge's answer to a real check is the ultimate observation of request-time enforcement.

The laboratory exposes this distinction in its demo:

```
cd companion
python3 -m logbranik demo
```

The command prints an initial allow, an ingestion result, a monitor outcome, an enforcement outcome, and an allow after revocation. It uses a temporary directory and generated keys, so it does not alter an existing Nginx configuration or retain credentials after completion.

The two paths

The request path contains Nginx, a local authorization check, and the protected application. It should remain usable when the central analyzer is unavailable. The analysis path contains logs, transport, storage, detection, and policy delivery. It is slower, retryable, and permitted to touch disks and databases.

Moving a database query into every authorization check joins those paths. A database maintenance operation then becomes an application outage. Moving signature verification into every check creates unnecessary CPU work and makes verification latency part of every request. Perform those operations when a policy changes. Publish a complete in-memory view afterward.

The architecture diagram in the exported edition shows both paths. Arrows represent actual information transfers, not an assumption that all transfers succeed. Every arrow across a machine boundary needs authentication, a maximum size, a timeout, and a definition of success.

Write the first invariants

An invariant describes behavior that must remain true despite retries and failures. For this system, expired decisions must stop denying requests even while central services are offline. A replayed older policy must not resurrect a revoked decision. A collector must not choose another node's identity by putting it in a JSON field. A repeated event must not be counted as additional evidence. A lost acknowledgement must not force a decision's lifetime to restart.

These statements are more useful than a feature list because they tell you what to test. "Supports retries" says little. "An exact duplicate policy returns its existing acknowledgement without reapplying or extending bans" defines a reproducible behavior.

Exercise

Write a timeline for five suspicious requests, an event retry, a central decision, a failed delivery, and a later successful delivery. Mark the earliest request that could be denied. Which interval should an operational alert measure?

Answer

The five original requests finish before their access observations exist. A retry adds no evidence after deduplication. The earliest denied request is one arriving after the agent has published the policy, assuming the site is in enforce mode and the decision has not expired. Alert on excessive desired-to-applied delay and on collection lag independently. Combining both delays into one number hides the failing component.

CC BY 4.0 / code MIT - candidate 0.2

2. Draw the trust boundaries

Begin with actors, not ports

A visitor requests application content. A collector sends observations from an edge. A publisher distributes policy. An agent evaluates policy locally. An operator changes modes, allowlists, and manual decisions. Each actor needs a distinct authority. A valid certificate is proof of possession of a credential; it does not, by itself, prove permission to perform every action.

The companion mTLS server therefore checks a certificate fingerprint against an explicit role registry after the TLS handshake. The collector credential is accepted at ingestion. The publisher credential is accepted at policy updates. A different certificate signed by the same laboratory CA cannot publish just because its chain validates. The transport tests include this exact case.

In a managed deployment you will probably use a certificate identity with a registration database rather than manually maintained fingerprints. Preserve the second authorization step. Transport identity must map to a role and allowed scopes. Do not derive role from a visitor-controlled header or from a body field that happens to say `collector`.

The local edge boundary

Nginx sends a deployment-configured site ID and a resolved client IP to the authorization handler. It overwrites the headers carrying those values. The handler must not receive arbitrary visitor headers or request bodies. Its socket permissions restrict which local users can connect.

A Unix socket avoids opening another TCP port, but it is not magical authentication. A compromised process with access to the socket can submit checks. A compromised edge root account can change Nginx configuration, replace agent state, or bypass the service entirely. This design does not claim protection against a hostile administrator of the enforcement host.

The lab has a loopback TCP option for hosts that cannot create Unix sockets. That option allows other local processes to connect and is for teaching. Use the Unix profile and operating-system permissions for the production boundary; verify it on the actual host.

Central observations are scoped

Suppose collector A is registered for `site-demo`. Its certificate maps to `node-a` and that site. A body containing `site-other` is not permission to ingest for another site. A body containing a fake node ID should be rejected by the schema, which has no node field. In the companion server, node identity comes from the certificate registry and is passed separately to the ingestion function.

The event schema is strict because accidental flexibility becomes authority. Ignoring unknown fields permits an old client to believe a new field is being enforced. Interpreting unknown site IDs as global scope creates cross-site consequences. Treat unknown versions, sites, and mutation types as explicit errors.

Decisions are data

A remote policy may describe an exact IP, a site, an expiry, and a reason code. It may not describe a shell command, executable path, firewall expression, arbitrary URL, or file to overwrite. The agent does not translate strings from a policy into commands. It evaluates a data structure.

This gives the control-plane compromise a defined blast radius. Someone who steals a signing key can still issue harmful valid decisions within the protocol's scopes and limits. They cannot obtain arbitrary code execution merely by adding a command field. Independent TTL caps, a local emergency bypass, and pinned site scopes reduce that damage without pretending it disappears.

A small threat table

Situation	Boundary that matters	Required response
Visitor spoofs guard headers	Nginx to agent	Overwrite identity headers
Collector sends another site	Ingestion authorization	Reject or quarantine scoped record
Valid CA certificate has wrong role	mTLS application authorization	Deny endpoint access
Old signed policy is replayed	Agent revision state	Reject lower revision
Signed policy has excessive TTL	Independent agent validation	Reject the policy
Root on the edge is hostile	Host administration	Outside this protocol's guarantee

Exercise

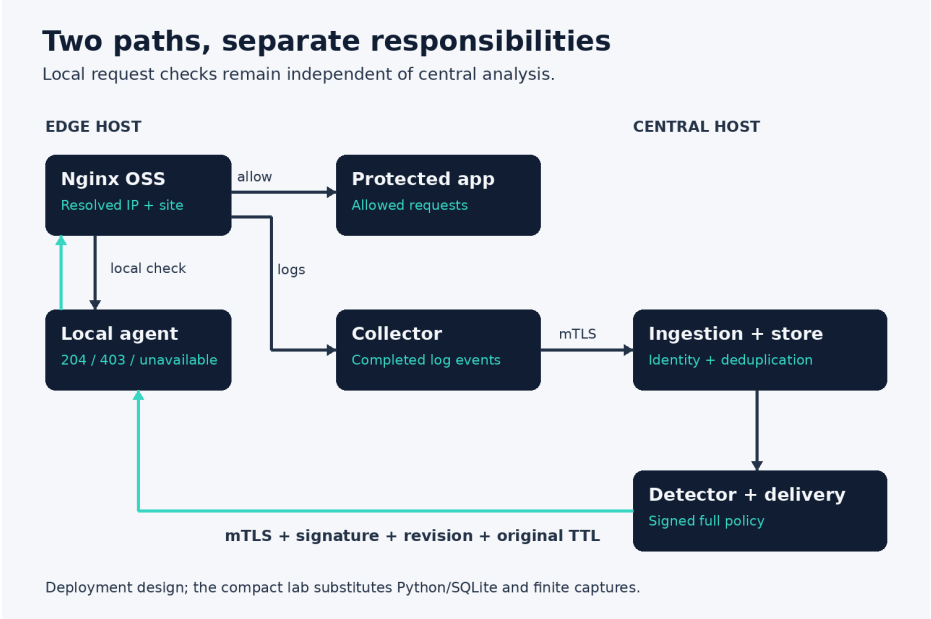
You are told to save time by sharing one certificate between collector, publisher, and operator. Explain the failure this creates and identify one test that would expose it.

Answer

A stolen collection credential would gain mutation rights, so a less privileged component could become an enforcement controller. Use separate credentials and role authorization. Connect to the snapshot endpoint using a CA-valid collector certificate and require denial. The companion's wrong-role test demonstrates the same boundary with a dedicated wrong-role certificate.

CC BY 4.0 / code MIT - candidate 0.2

3. Design around failure



Two paths: local authorization and central analysis.

Two machines, different responsibilities

Edge A runs Nginx, the collector, and an agent. Central B runs ingestion, durable storage, a detector, and delivery. The edge's request checks do not call Central B. The central side can inspect events and prepare policy without being in the path of every application request.

At a larger scale, several edges belong to one organization and serve registered sites. Each agent receives only its assigned sites. The initial design has one organization boundary. Adding customer tenancy later changes authorization, data retention, key ownership, and database constraints. Treat it as a new design task rather than adding a `tenant_id` field and declaring isolation solved.

Central outage

An already synchronized agent has a saved policy. If Central B becomes unavailable, the agent continues evaluating that policy. Each ban ends at its original expiry. The collector accumulates events only within its configured finite capacity. Once capacity is exhausted, behavior depends on the collector's backpressure and retention configuration; there is no infinite queue.

An outage therefore affects new knowledge, delivery, and stored observations. It does not necessarily affect ordinary request availability. The important exception is a first-boot agent without verified state. That agent cannot honestly say it has synchronized an empty policy. It reports not-ready until it receives a valid snapshot, including a signed empty one.

Agent outage

An absent local agent is a different case. The baseline protected Nginx location treats authorization errors as a failure. An allow response is 204. A deliberate deny is 403. A backend error is not an allow. Depending on configuration and build, the public result of an authorization error is generally an Nginx error response, often 500.

Some sites need a deliberate availability profile that permits requests if the local guard is down. That is a business decision about the protected application. Implement it inside the internal authorization location, where only selected backend error codes can be converted to an allow response. Never put a broad fallback on the public application location: it could transform a real application error or a real deny into success.

The production-design folder contains the optional fail-open example from the source pack. It is a candidate configuration until a real Nginx test verifies its status handling, request method, body preservation, and location routing. Do not enable it because the text looks plausible.

Database and filesystem failures

Ingestion returns success after its transaction commits. If storage fails, the response must invite a retry instead of pretending the data was saved. Policy publication similarly acknowledges only after persistent state and the live view agree. The agent test injects a disk-write failure and verifies that the previous view remains published and survives a new agent object reading the existing state.

A crash between persistence and live publication creates a short uncertainty window. On restart, the verified saved state becomes the source of truth. The sender may retry the exact policy and receive an idempotent acknowledgement. A crash before persistence means the sender has no valid acknowledgement and must try again.

Filesystem permissions also matter. State belongs in a directory an unprivileged service can write without granting arbitrary application users access. Replacing a file in a protected directory is different from writing through a path an attacker can redirect. Configure the directory deliberately and avoid world-writable runtime paths.

Availability is a policy

Do not label a system "fail-safe" without naming the failure. A central outage and an agent outage have different consequences. Monitor mode and emergency bypass have different purposes. Clock uncertainty and an ordinary expired ban must not be treated as the same event. Document each response in a small failure matrix and test the responses that matter to the deployment.

Exercise

An agent is offline from central services for thirty minutes. It previously applied a fifteen-minute ban. What should happen to requests from that IP after minute fifteen? What happens if the agent process itself disappears at minute five?

Answer

The live agent allows the address at its original expiry. A missing agent produces an authorization error under the baseline profile. These failures need separate alerts and recovery actions.

4. Run the laboratory

Set up an isolated working directory

Use Python 3.12 or a compatible newer release on Linux for the full laboratory. The preparation run used Python 3.12 and Cryptography 50.0.1. The requirements file provides a compatible major-version range, while the verification report identifies the version actually used. Reproduce tests with that recorded version before upgrading dependencies.

From the extracted book package:

```
cd companion
python3 -m venv .venv
. .venv/bin/activate
python3 -m pip install -r requirements.txt
python3 -m unittest discover -s tests -v
python3 -m logbranik demo
```

The demo runs in a temporary directory. Its stable outcomes are:

```
before (204, 'allow')
ingest {'accepted': 5, 'duplicates': 0, 'quarantined': 0}
monitor (204, 'would_block')
enforce (403, 'block')
revision 3
revoked (204, 'allow')
```

The request ID and decision ID are implementation data, not secrets in this example. Do not turn the example's fixed IP into a real-world allowlist or threat label. It is a documentation address used to make policy transitions reproducible.

Move from functions to processes

Initialize persistent lab state and create disposable loopback certificates:

```
python3 -m logbranik init
python3 -m logbranik certificates
```

These commands refuse to overwrite an existing state or certificate directory. The state directory contains a central signing key and an agent's public verification key because this is a single-machine lesson. In a two-machine deployment, copy only the public key, agent

identity, provisioned epoch, and authorized-site configuration to the edge. The central signing private key must not travel with it.

In terminal A, start the agent:

```
python3 -m logbranik agent
```

If your teaching host prohibits AF_UNIX sockets, use `python3 -m logbranik agent --tcp-auth`. That binds the authorization handler to loopback port 9101. It does not change the mTLS control listener, which uses loopback port 9443.

In terminal B, start ingestion:

```
python3 -m logbranik central-server
```

In terminal C, start periodic delivery:

```
python3 delivery_loop.py
```

The delivery process computes a full policy and sends it over mTLS. It checks the returned revision and payload hash. Its one-second loop is a teaching mechanism. The production design replaces repeated whole-state generation with durable desired-state changes, coalescing, a delivery outbox, and independent reconciliation.

Feed real schema-shaped events

Generate local fixture events without accessing another server:

```
python3 - <<'PY'
import time
from pathlib import Path
from logbranik.__main__ import fixture_event
from logbranik.protocol import encode
Path('events.ndjson').write_bytes(
    b'\n'.join(encode(fixture_event(i,time.time())) for i in
    range(1,6)))
PY
```

Send them through the mTLS ingestion endpoint:

```
curl --cacert lab-state/tls/ca.crt \
  --cert lab-state/tls/collector.crt \
  --key lab-state/tls/collector.key \
  -H 'Content-Type: application/x-ndjson' \
  --data-binary @events.ndjson \
  https://localhost:9444/v1/events:batch
```

Inspect decisions with `python3 -m logbranik decisions`. Switch the registered site to enforce mode with `python3 -m logbranik mode enforce`. The delivery loop carries the updated mode to the agent. Check through the Unix socket:

```
curl --unix-socket lab-state/auth.sock -i \
  -H 'X-Guard-Site: site-demo' \
  -H 'X-Guard-IP: 198.51.100.23' \
  http://localhost/v1/check
```

For the TCP teaching profile, use the same headers at `http://127.0.0.1:9101/v1/check`. Confirm a 403 only after enforcement mode arrives. A 204 with `would_block` is correct in monitor mode.

What this proves

The function demo proves the central-to-agent logic in one process. The process exercise proves transport, credentials, persistence, and live publication. Neither proves that Nginx calls the handler for every protected route. Lesson 8 adds that deployment gate.

Exercise

Submit the same `events.ndjson` twice. Then edit one field in a record while preserving its request ID and submit it again. Compare the ingestion counters.

Answer

The first retry is reported as duplicates and adds no evidence. The conflicting record is quarantined; it does not overwrite the original. The laboratory saves only the quarantine reason, avoiding a second raw copy of potentially sensitive input. A production quarantine should also have explicit retention, capacity, and access controls.

CC BY 4.0 / code MIT - candidate 0.2